

Hola Rubén,

He estado revisando tu proyecto de OpenPanel y quería darte feedback porque la verdad es que está muy bien.

Vamos por partes.

Lo primero, lo que has hecho muy bien:

La arquitectura GitOps que has montado es sólida. Me gusta mucho la separación que has hecho entre el repo de la aplicación y el de infraestructura, y cómo los conectas con el `repository_dispatch`. Eso es exactamente cómo se suele hacer en producción. El hecho de que el fork de OpenPanel construya las imágenes y luego notifique a este repo para que actualice los tags demuestra que entiendes bien el flujo end-to-end.

El blue-green con Argo Rollouts está muy bien pensado. Me parece correcto que hayas deshabilitado la promoción automática (`autoPromotionEnabled: false`) porque en producción nunca quieres que un despliegue se promocione sin que alguien haya verificado primero que funciona. Y el detalle de mantener el ReplicaSet anterior caliente durante 10 minutos (`scaleDownDelaySeconds: 600`) para que el rollback sea instantáneo... eso demuestra que has pensado en los escenarios reales de operación.

El stack de observabilidad es completo. No solo has montado Prometheus, Loki y Tempo, sino que los has integrado bien en Grafana. Los tres dashboards que has creado (OpenPanel, cluster, logs+traces) son útiles y están bien enfocados. Las alertas que has definido tienen sentido: APIDown, HighErrorRate, latencia P99, lag del event loop... son métricas que realmente importan para operar este tipo de aplicación.

Lo de Sealed Secrets está muy bien resuelto. El script `ensure-sealing-key.sh` que guarda el backup en `~/.config/openpanel/sealing-key.yaml` es importante porque sin eso, si pierdes el clúster, pierdes todos los secretos cifrados. Es un detalle que mucha gente olvida.

De momento eres la única persona que ha incluido la documentación en docs/final-project.md: diagramas, explicaciones de arquitectura, referencias... se nota que has dedicado tiempo a documentar no solo qué has hecho, sino por qué lo has hecho así. Eso es lo que diferencia un proyecto que funciona de uno que se puede mantener y escalar.

Los módulos de Terraform están bien estructurados. backup-storage, iam-user, iam-irsa... cada uno con su responsabilidad clara. Y el diseño de arquitectura AWS con el diagrama de VPC multi-AZ y el desglose de costes (~525€/mes) demuestra que has pensado en cómo se desplegaría esto en un entorno cloud real, no solo en Minikube.

Ahora, donde veo espacio para mejorar:

Sobre los backups:

Lo bien hecho:

- Tienes las dos capas correctas: Velero para recursos K8s y backups nativos de cada BD (PostgreSQL, Redis, ClickHouse)
- PostgreSQL está bien porque `pg_dump` guarda el archivo en el host local

- Redis con `SAVE` está correcto

El problema con ClickHouse: Tu script crea el backup dentro del pod con `BACKUP DATABASE TO File(...)`, pero nunca saca el archivo fuera. Si el pod se elimina o el PVC se corrompe, pierdes el backup. A diferencia de PostgreSQL, el archivo de ClickHouse no sobrevive fuera del contenedor.

Añade un `kubectl cp` después de crear el backup para copiar el `.zip` fuera del pod, o usa un volumen compartido.

Es un fix sencillo pero importante. ClickHouse es crítico para OpenPanel (todos los eventos de analítica), así que necesitas poder recuperar esos backups incluso si el pod desaparece.

Otra cosa: no veo evidencia de tests automatizados de integración o E2E en los workflows. Tienes validación de manifests con kubeconform y kube-linter, que está muy bien, pero no hay tests que verifiquen que la aplicación realmente funciona después de desplegarse. Algo simple como smoke tests que verifiquen que la API responde, que el dashboard carga, que se puede crear un usuario... eso daría más confianza en los despliegues automatizados.

En resumen:

Has entregado un proyecto muy sólido. No solo has cumplido con los requisitos del feedback anterior (diagrama de AWS, Terraform, dashboards, alertas, Sealed Secrets, plan de trabajo), sino que has ido más allá con detalles como el backup de la sealing key, la integración Loki+Tempo en Grafana, y la arquitectura de CI/CD con repos separados.

Es un proyecto production-ready con el que se podría aprender mucho. Los puntos de mejora que te menciono son sutilezas. Nada que impida que el proyecto funcione, pero cosas que lo llevarían al siguiente nivel si quieres perfeccionarlo.

Enhorabuena por el trabajo.

Un saludo!